

Lab3-2 实验报告：智能出行规划器

本次 Lab3-2 我做的是一个“规划—地点—天气—行程—AI 总结”的小系统。核心目标是把前后端分离整合成一个能闭环使用的产品：用户能创建规划、在地图上选点加入、看到天气、把地点安排到上午/下午/晚上，并在最后得到 AI 的总结建议。

1) 你的前端和后端分别负责了什么？

我把职责分得比较明确：

- **前端 (Vue 3) 负责：**
 - 页面交互与状态管理：规划列表/表单、地点列表、行程分配、AI 总结展示、导出按钮、规则检查结果展示
 - 地图交互：加载高德地图 JS API，点击选点、打 marker、调用逆地理编码拿地址与 `adcode`
 - 只与后端 `/api` 交互，不直接请求天气/LLM 的第三方接口
- **后端 (FastAPI) 负责：**
 - 数据持久化：SQLite 中保存 `plans` / `places` / `itinerary_items`
 - 业务 API：规划 CRUD、地点增删查、行程保存与读取
 - 第三方服务代理：统一调用高德天气 (Web 服务)，统一调用 OpenAI 兼容的 LLM 接口
 - 安全与配置：天气 Key、LLM Key 存在 `backend/.env` (本地文件，不提交)

这样做的好处是：前端更像“一个干净的 UI 客户端”，后端更像“唯一可信的数据与服务入口”，不容易把 Key 泄漏到浏览器侧。

2) 你是如何组织规划数据的？

我用 SQLite 做了三张表来组织规划数据，并围绕 `plan_id` 建立关系：

- **plans (规划)** : `id`, `title`, `date`, `budget`, `people_count`, `preferences`, `created_at`, `updated_at`
- **places (地点)** : `id`, `plan_id`, `name`, `address`, `lng`, `lat`, `adcode`, `note`, `sort_index`, `created_at`
- **itinerary_items (行程项)** : `id`, `plan_id`, `place_id`, `time_slot`, `sort_index`, `created_at`

不管是地点还是行程，都挂在同一个规划下面，API 也按 `/api/plans/{plan_id}/...` 组织，结构清晰。并且前端的状态只是展示用，真正的“当前规划数据”都以数据库为准，这样重启后端或刷新页面后数据仍然一致。

3) 你如何处理天气信息和规划之间的关系？

天气是和地点相关的动态信息。我用了下面这种关联方式：

- **地点保存 `adcode`**：地图选点后逆地理编码拿到 `adcode`，把它和地点一起保存到 `places` 表里。
- **后端统一查询天气：**
 - `GET /api/weather/live?adcode=...`：按单个 `adcode` 查实时天气
 - `GET /api/plans/{plan_id}/weather/live`：遍历规划内地点（有 `adcode` 的才查），按 `place_id` 聚合返回天气字典
- **前端只展示**：地点列表与“选点面板”都展示天气摘要（状态、温度等），不会直接拿 Key 去请求高德。

天气会变化，不适合持久化为规划的固定字段；但是把 `adcode` 存下来，又能让天气查询稳定绑定到具体地点。

4) AI 在系统中的角色是什么，为什么它不是主角？

AI 在这个系统里更像“最后一步的辅助总结器”，而不是主角：

在用户已经完成“规划/地点/行程”的基础上，读入这些上下文（地点、天气、时间段、偏好等），输出更可执行的建议，比如顺序建议、注意事项、雨天备选等。

1. 规划管理、地点管理、行程安排必须稳定可验收；AI 很容易受影响，不稳定。
2. 如果没有结构化数据（地点、天气、时间段），AI 只能泛泛而谈；所以系统的主角应该是数据与流程，AI 是锦上添花。

5) 在这个实验中，你觉得最难的是前端、后端、数据管理，还是前后端协作？为什么？

我觉得最难的是前后端协作

1. 边界划分要一开始就想清楚：比如天气与 LLM 的 Key 放哪里、哪些接口该后端代理、前端要不要缓存，这些如果前期不清楚，后面会反复重构。
2. 前端要处理加载中/失败/空状态，后端要把第三方错误变成可读的 `detail` 返回。实际联调时经常出现“配置/网络/模型名/URL”不对。
3. 数据一致性：地点加入、行程保存、天气刷新、规则检查刷新这些动作之间要保持顺序。

相比之下，单独写前端或单独写后端都没那么难，难的是把它们当成一个系统跑通并稳定展示结果。